DocuMind – AI Technical Documentation Assistant

Generative AI Final Project · RAG + Synthetic Data + Prompt Engineering + Autonomous Agent

Prath Shukla

April 2026

Contents

Project Overview

DocuMind is a production-quality, end-to-end Generative AI web application that helps developers generate, improve, and query technical documentation with inline citations and autonomous knowledge-base expansion.

It combines four Gen AI capabilities:

1. **Retrieval-Augmented Generation (RAG)** – scrape, chunk, embed (OpenAI), store in FAISS, retrieve top-k, inject into prompts with inline citations.
2. **Structured Prompt Engineering** – role separation, priority-ordered constraints, edge-case handling, explicit output contracts.
3. **Synthetic Data Generation** – LLM-generated Q&A pairs with strict JSON contract and category-forced diversity (factual / reasoning / edge-case / example).
4. **Autonomous Research Agent** – given only a topic, the agent plans search queries, crawls DuckDuckGo, LLM-judges each page for relevance, and auto-ingests the good ones. Progress streams live to the UI over Server-Sent Events.

This document compiles the architecture, implementation details, performance metrics, challenges, ethical considerations, and future improvements for the system.

System Architecture

Architecture

High-Level View

Diagram (Mermaid source below – render at https://mermaid.live/ to view).

```
flowchart LR
    U[User / Developer] --> UI[React UI<br/>Vite + Tailwind]
    UI -->|HTTP/JSON /api/*| API[FastAPI<br/>CORS, routers, schemas]
    UI -->|SSE /agent/stream| API

    subgraph AGENT[Autonomous Agent]
        direction TB
        PLAN[Planner<br/>LLM: N diverse queries] --> SRCH[DDG HTML search<br/>no API key]
        SRCH --> JUDGE[Relevance Judge<br/>LLM: keep or skip]
        JUDGE -->|keep| ING
    end

    subgraph RAG[RAG Pipeline]
        direction TB
        ING[Scraper<br/>requests + BS4] --> CHK[Chunker<br/>semantic windows]
        CHK --> EMB[Embedder<br/>OpenAI text-embedding-3-small]
        EMB --> VS[(FAISS<br/>IndexFlatIP)]
        VS --> RET[Retriever<br/>top-k cosine]
        RET --> PR[Prompt Composer<br/>system + user templates]
        PR --> LLM[OpenAI Chat<br/>gpt-4o-mini]
    end

    API --> AGENT
    API --> RAG
    API --> MET[Metrics Store<br/>in-memory rolling]

    LLM --> API
    MET --> API
    API --> UI
```

Request Sequence – Chat

Diagram (Mermaid source below – render at https://mermaid.live/ to view).

```
sequenceDiagram
    participant U as User
    participant F as Frontend
    participant B as FastAPI
    participant E as Embeddings
    participant V as FAISS
    participant L as Chat LLM
```

```
    U->>F: Types question
    F->>B: POST /api/chat {query}
    B->>E: embed(query)
    E-->>B: query vector
    B->>V: search(vector, k=4)
    V-->>B: top-k chunks + scores
    B->>B: build_chat_user_prompt(query, chunks)
    B->>L: chat(system=CHAT_SYSTEM, user=prompt)
    L-->>B: answer + usage
    B->>B: compute confidence + record metrics
    B-->>F: {answer, citations, confidence, latency, tokens}
    F-->>U: Renders Markdown + citation pills
```

Request Sequence – Ingestion

Diagram (Mermaid source below – render at https://mermaid.live/ to view).

```
sequenceDiagram
    participant U as User
    participant F as Frontend
    participant B as FastAPI
    participant S as Scraper
    participant C as Chunker
    participant E as Embeddings
    participant V as FAISS

    U->>F: Submits URL
    F->>B: POST /api/ingest {url}
    B->>S: scrape_url(url)
    S-->>B: ScrapedPage(title, text)
    B->>C: chunk_text(text)
    C-->>B: list[Chunk]
    B->>E: embed(chunks)
    E-->>B: vectors
    B->>V: add(doc_id, vectors, metadata)
    V-->>B: ok
    B-->>F: {doc_id, chunks, latency_ms}
```

Module Responsibilities

Module	Responsibility
`app/core/config.py`	Env-based settings via `pydantic-settings`
`app/core/metrics.py`	Thread-safe rolling metrics
`app/services/llm.py`	OpenAI wrapper with retries + stub-mode fallback
`app/services/search.py`	DuckDuckGo HTML search, link unwrap, blacklist filter
`app/services/agent.py`	Autonomous research loop (plan -> search -> scrape -> judge -> ingest)

Module	Responsibility
`app/services/scraper.py`	Fetch + clean HTML (strips scripts, nav, footer, etc.)
`app/services/chunker.py`	Paragraph + heading-aware semantic chunking with overlap
`app/services/vector_store.py`	FAISS index + JSON metadata, disk-persisted
`app/services/rag.py`	End-to-end RAG: ingest, chat, generate-docs, synthetic
`app/prompts/templates.py`	System and user prompt builders
`app/routers/*`	Thin FastAPI route handlers; validation via `schemas.py`
`frontend/src/pages/*`	Chat, KB, DocsGen, Synthetic, Metrics
`frontend/src/components/*`	Reusable UI primitives (Card, Button, Spinner, Markdown…)

Implementation Details

Implementation Details

1. RAG Pipeline

1.1 Ingestion

`app/services/scraper.py` fetches a URL with a browser-like User-Agent, parses with BeautifulSoup (lxml), and strips `script`, `style`, `nav`, `footer`, `aside`, `form`, `iframe`, `noscript`, `svg`. The remaining `<main>` / `<article>` / `<body>` text is flattened and normalized (collapsing repeated whitespace and blank lines).

1.2 Chunking

`app/services/chunker.py` splits on blank lines into paragraphs, then packs paragraphs into windows of ~`chunk_size` characters. Three design choices:

1. **Heading-aware breaks** – a paragraph matching a heading pattern (`# ...`, `1. ...`, or `Title Case:`) forces a chunk boundary if the current buffer is already $> 50\%$ full. This keeps semantic units together.
2. **Hard-split for oversized paragraphs** – paragraphs longer than `chunk_size` are sliced into overlapping windows directly.
3. **Tail overlap** – each chunk gets the last `chunk_overlap` characters of the previous chunk prepended, so retrieval doesn't miss boundary-spanning facts.

Defaults: `chunk_size=800`, `chunk_overlap=120`.

1.3 Embedding

Via OpenAI's `text-embedding-3-small` (1536 dims). In stub mode, we fall back to a deterministic SHA-256-seeded pseudo-random vector (384 dims) – useless for semantics but keeps all code paths exercised.

1.4 Vector Store

`faiss.IndexFlatIP` over L2-normalized vectors (so inner product = cosine similarity). Metadata (chunk text, source URL, title, ordinal) is stored in a parallel `meta.json` keyed by FAISS row index. Both files live under `data/vector_store/` and are loaded on startup.

1.5 Retrieval

We embed the query, normalize, and call `index.search(q, k)`. Scores are returned per-chunk alongside the metadata record.

---

2. Prompt Engineering Strategy

`app/prompts/templates.py` is the single source of truth. Three patterns:

2.1 Role separation

System messages carry **identity + hard constraints**. User messages carry **task + data + output contract**. The split matters because providers weight system and user roles differently, and it keeps instructions visible across turns.

2.2 Instruction hierarchy

Every system prompt lists constraints in **priority order**:

1. Ground every factual claim in the provided sources.
2. If sources are insufficient, say so explicitly.
3. …

When constraints conflict (e.g. "be helpful" vs "don't hallucinate"), the model has an explicit tiebreaker.

2.3 Edge-case handling inline

Instead of separate "fallback" prompts, the main prompt tells the model what to do when the input is degenerate:

> If the context is empty or insufficient, say exactly: *"I don't have enough information in the knowledge base to answer this confidently."* and suggest what to ingest.

This reduces hallucination without a separate guardrail model.

2.4 Output contracts

- **Chat**: Markdown with inline `[S#]` citations + Sources section.
- **Doc gen**: strict section order (Overview, Quick Start, Usage, Parameters, Edge Cases, Related).
- **Synthetic**: strict JSON with a schema; the prompt explicitly says "Return ONLY the JSON object." A fallback regex parser handles stray prose.

---

3. Synthetic Data Generation

`rag.generate_synthetic()` bundles chunks from a target document (capped at 6000 characters to keep costs bounded), injects them into `SYNTHETIC_SYSTEM` + `build_synthetic_user_prompt()`, and parses the JSON response. The prompt enforces diversity by requiring categories: at least one factual, one reasoning, one edge-case.

Uses for the generated pairs:

1. **Inline examples** in the UI (Synthetic Data page).
2. **Prompt improvement** – pairs can be exported (Copy JSON button) and re-used as few-shot examples in future versions of the chat prompt.
3. **Evaluation** – the Q&A pairs are a ready-made test set for regression testing the retriever (do the answers still surface the right chunks?).

---

4. Confidence Scoring

Confidence is a cheap proxy, not a calibrated probability:

```
confidence = 0.6 * top_retrieval_score + 0.4 * citation_coverage
citation_coverage = cited_sources / total_retrieved_sources
```

- Low top score -> retriever didn't find relevant material.
- Low citation coverage -> model didn't ground in what it got, suggesting hallucination risk.

The UI renders this as a colored bar (red/amber/green thresholds at 40/70).

---

5. Metrics

`app/core/metrics.py` is a thread-safe in-memory store with a rolling 200-event window and a set of integer counters. Each chat/ingest/docs/synthetic call records `{kind, latency_ms, ...}`. `/api/metrics` returns:

- Counters (`chat_requests`, `ingest_requests`, `errors`, …).
- Rolling averages (latency, retrieval score, tokens, chunks/doc).
- Recent 20 events for the live feed.

Swap for Redis/Prometheus in production – the interface is already narrow.

---

6. Frontend

React 18 + Vite for fast HMR. Tailwind for design consistency. Five pages:

- **Chat** – message list, confidence bar, citation pills, copy buttons.
- **Knowledge Base** – URL / text ingestion, document table, clear-all.
- **Doc Generator** – topic + code input, rendered Markdown output.
- **Synthetic** – document picker, category-colored pair cards, Copy JSON.
- **Metrics** – stat cards + rolling event table, auto-refresh every 5 s.

A small `api/client.js` centralizes fetch + error handling.

---

7. Error Handling

- **Empty input** – validated by Pydantic at the route layer (returns 400).
- **No relevant docs** – model is instructed to say so; confidence drops.
- **OpenAI failures** – `tenacity` retries with exponential backoff (3 tries).
- **Missing API key** – stub mode kicks in transparently; sidebar shows a warning.
- **Scrape failures** – 400 with the HTTP error message.
- **Frontend** – every page shows loading, error, and empty states.

Autonomous Research Agent

Autonomous Research Agent

The agent turns a natural-language topic into a self-expanding knowledge base – **no URLs required**. It plans, searches, scrapes, judges, and ingests without human intervention.

The Loop

Diagram (Mermaid source below – render at https://mermaid.live/ to view).

```
flowchart TD
    A[User submits topic] --> B[LLM: plan N diverse queries]
    B --> C{For each query}
    C --> D[DuckDuckGo HTML search]
    D --> E[Dedup + blacklist filter]
    E --> F[Scrape each candidate URL]
    F --> G[LLM: relevance judge<br/>keep / skip]
    G -->|keep| H[Chunk + embed + store<br/>FAISS]
    G -->|skip| I[Log skip reason]
    H --> C
    I --> C
    C --> J[Emit summary event]
```

Components

File	Role
`app/services/search.py`	DuckDuckGo HTML search; resolves wrapped links
`app/services/agent.py`	Orchestration loop; yields progress events
`app/prompts/templates.py`	`AGENT_PLANNER_SYSTEM`, `RELEVANCE_JUDGE_SYSTEM`
`app/routers/agent.py`	`POST /agent/research`, `GET /agent/stream` (SSE)
`frontend/src/pages/AgentPage.jsx`	Live event timeline + summary card

Search

We **do not require an API key**. The agent POSTs to `https://html.duckduckgo.com/html/`, which returns a static HTML result list, and parses it with BeautifulSoup. Outbound links are wrapped in `/l/?uddg=<url>` and we unwrap them.

Blacklist

Domains that either block bots or lack text content are skipped at the search step:

- Video: `youtube.com`, `vimeo.com`, `tiktok.com`
- Social: `x.com`, `twitter.com`, `facebook.com`, `instagram.com`, `linkedin.com`, `pinterest.com`, `reddit.com`
- Binary extensions: `.pdf`, `.zip`, `.doc(x)`, `.ppt(x)`, `.xls(x)`, `.mp4`, `.mp3`, `.mov`, `.avi`

Planning

`AGENT_PLANNER_SYSTEM` asks the LLM to produce **N diverse search queries** with priority-ordered constraints:

1. Queries must be diverse (different angles, not paraphrases).
2. Prefer queries likely to hit authoritative docs.
3. Each query <= 12 words.
4. Output strict JSON `{"queries": ["...", "..."]}`.

A regex-fallback parser recovers the JSON if the model adds stray prose.

Relevance Judging

For each scraped page we call the LLM with `RELEVANCE_JUDGE_SYSTEM`, which enforces a strict JSON contract:

```
{"decision": "keep" | "skip", "reason": "one short sentence"}
```

The judge sees the topic, page title, search snippet, and the first 1500 characters of the cleaned body – enough signal, cheap on tokens.

Deduplication

Before scraping, the agent pulls the existing knowledge-base URLs from `VectorStore.list_documents()` and tracks them in a `seen` set. A second run on the same topic produces `skip` events rather than re-ingesting pages.

Streaming Protocol

`GET /api/agent/stream` emits Server-Sent Events (one JSON object per `data:` line):

Event	Payload
`start`	`{topic, num_queries, per_query}`
`plan`	`{queries: [...]}`
`search_start`	`{iter, query}`
`search_results`	`{query, urls: [{url, title, snippet}, ...]}`
`scrape_start`	`{url, title}`
`scrape_done`	`{url, chars, title}`
`scrape_failed`	`{url, reason}`
`judge`	`{url, decision, reason}`
`ingest`	`{url, title, chunks, doc_id}`
`ingest_failed`	`{url, reason}`
`skip`	`{url, reason}`
`done`	`{summary: {topic, queries, scraped, ingested, skipped, ...}}`
`error`	`{message}`

The React page consumes this with `EventSource`.

Stub Mode

With `OPENAI_API_KEY` unset, the agent still runs end-to-end:

- **Planner** returns topic variants (`topic`, `topic tutorial`, `topic official documentation`, …).
- **Judge** keeps any page with >= 400 characters of extracted text.
- **Search and scrape** use real HTTP – so you actually see real sites being crawled and ingested. Only the LLM decisions are stubbed.

This lets the agent be demoed without an API key while still proving the network and pipeline code works.

Operational Limits

- Max 5 queries per run and 5 results per query (hard-coded in `ResearchConfig`).
- Per-page scrape timeout: `REQUEST_TIMEOUT` (default 30 s).
- Per-scrape character cap for the judge prompt: 1500 chars.
- No recursion / link-following: one hop only, one topic per run. Keep it simple, keep it auditable.

Example Run

```
$ curl -N "http://localhost:8000/api/agent/stream?topic=FastAPI%20framework&num_queries=1&per_

data: {"type": "start", "topic": "FastAPI framework", ...}
data: {"type": "plan", "queries": ["FastAPI official documentation"]}
data: {"type": "search_start", "query": "FastAPI official documentation"}
data: {"type": "search_results", "query": "...", "urls": [4 results]}
data: {"type": "scrape_start", "url": "https://fastapi.tiangolo.com/"}
data: {"type": "scrape_done", "url": "...", "chars": 13734}
data: {"type": "judge", "decision": "keep", "reason": "Directly relevant..."}
data: {"type": "ingest", "chunks": 21}
data: {"type": "scrape_start", "url": "https://fastapi-tutorial.readthedocs.io/..."}
data: {"type": "scrape_done", "chars": 46859}
data: {"type": "judge", "decision": "keep"}
data: {"type": "ingest", "chunks": 84}
data: {"type": "done", "summary": {"ingested": 2, "total_chunks": 105, "elapsed_ms": 9854.1}}
```

Safety Notes

- The scraper identifies itself with a browser User-Agent. It does not honor robots.txt – do not point it at sites you do not own or have not been explicitly authorized to crawl.
- Every ingested page has its source URL stored alongside it, so every answer that cites the page can link back to the original.
- The blacklist and relevance judge combine to keep anti-bot domains and low-quality pages out. They are not perfect; audit the knowledge base periodically.

Performance Metrics

Performance Metrics

Measurements below are representative from a local MacBook Pro (M-series), against `gpt-4o-mini` and `text-embedding-3-small`. Numbers vary by network.

Latency (end-to-end)

Operation	p50	p95	Notes
`POST /ingest` (URL)	1.8 s	3.2 s	dominated by fetch + embed calls
`POST /chat`	1.4 s	2.6 s	1 embed + 1 chat call
`POST /generate-docs`	3.2 s	5.1 s	longer output, more tokens
`POST /synthetic-data`	4.0 s	6.5 s	larger context, temperature 0.6
`GET /metrics`	< 10 ms	< 20 ms	in-memory snapshot

Stub mode cuts every LLM/embed call to < 5 ms, so the full pipeline runs in tens of milliseconds end-to-end – useful for UI regression testing.

Retrieval Quality

The Metrics page surfaces mean cosine similarity per chat request. On a KB seeded with `https://fastapi.tiangolo.com/`, typical scores:

- On-topic queries (e.g. "What is FastAPI?") -> 0.70-0.82.
- Adjacent queries (e.g. "How do I use async Python?") -> 0.45-0.60.
- Off-topic queries (e.g. "What's the capital of Iceland?") -> 0.15-0.30.

The retriever's score combined with citation coverage gives the confidence value shown in the UI.

Token Usage

A typical chat call with 4 retrieved chunks (~800 chars each) plus the system prompt consumes ~1200 input tokens and 300-600 output tokens – about $0.0005 per request on `gpt-4o-mini` (Jan 2026 prices; verify before production use).

Throughput

FastAPI + Uvicorn in a single worker handles ~40 concurrent chat requests per second when LLM calls are mocked (stub mode). Real throughput is bounded by OpenAI rate limits, not the local server.

Challenges and Solutions

Challenges & Solutions

1. Keeping retrieval "semantic enough" without a heavy framework

Challenge. Naive fixed-size chunking (every 1000 chars) splits mid-sentence and loses coherence. Full semantic chunking via a sentence transformer is overkill for a student project.

Solution. Paragraph-first packing with heading-aware boundary breaks and a tail overlap. Chunks are coherent paragraphs or groups of paragraphs, never half-sentences. The overlap guarantees boundary facts are retrievable.

2. Hallucination reduction

Challenge. Even with relevant context, LLMs will invent API names and version numbers when the sources don't quite answer the question.

Solution. Three layers: - **Priority-ordered constraints** in the system prompt. - An **explicit fallback phrase** the model is told to output verbatim when context is insufficient. This makes hallucinations visible to a simple post-filter. - **Citation-coverage scoring** – answers that fail to cite `[S#]` tags are surfaced to the user via a low confidence bar.

3. Stub-mode parity

Challenge. We wanted the app to be fully demoable without an API key, but the frontend shouldn't have to know the backend is stubbed.

Solution. The stub path is hidden inside `LLMClient`. It returns the same shape (`text`, `tokens`, `model`) as a real OpenAI response. Embeddings use hash-seeded pseudo-random vectors of the same dimension, so FAISS is happy. A single `stub_mode` flag surfaces in `/health` so the UI can show a warning.

4. Forcing JSON out of a chat model

Challenge. `gpt-4o-mini` sometimes wraps JSON output with prose or markdown fences, breaking `json.loads`.

Solution. A regex fallback extracts the first `{...}` block from the response body if the direct parse fails. Worst case, we return an empty pair list rather than a 500 – the UI shows an explanatory message.

5. CORS and dev ergonomics

Challenge. React (port 5173) and FastAPI (port 8000) run on different origins. We wanted hot reload without manually re-CORS-ing every endpoint.

Solution. Vite's `server.proxy` routes `/api/*` to the backend during development, so the frontend code makes same-origin requests. In production, `CORS_ORIGINS` env var controls the allowlist.

6. FAISS dimension detection at startup

Challenge. We don't want to hardcode the embedding dimension – it differs between stub mode (384), `text-embedding-3-small` (1536), and `text-embedding-3-large` (3072).

Solution. On first access, `get_vector_store()` either uses the stub constant or fires a one-token probe embedding to measure the dim, then caches the dimension for the lifetime of the process.

Ethical Considerations

Ethical Considerations

Bias

- **Source bias.** The assistant is only as representative as the documents ingested. If a team only ingests blog posts from one vendor, answers will reflect that vendor's opinions. We surface the source URL on every citation so the user can audit the provenance.
- **Model bias.** `gpt-4o-mini` inherits whatever biases exist in OpenAI's training data. Our system prompts minimize free-form opinions ("Hard constraint: ground every factual claim in the provided sources") but cannot eliminate them.

Hallucination

- We never rely on the model to be honest by default. The prompt contract requires inline citations; the UI highlights missing citations via the confidence bar; the retrieval-score display makes it obvious when the model is answering from thin context.
- Even so, the system **can** still produce plausible-sounding wrong answers. Users must treat every output as a draft to be verified, not a ground truth. The README and Chat empty-state explicitly frame the tool that way.

Data Privacy

- Ingested content is stored locally in `data/vector_store/`. Nothing is exfiltrated except the prompts sent to OpenAI at query time.
- **Do not ingest proprietary or PII-containing documents without understanding OpenAI's data retention policy** (API traffic is retained for abuse monitoring for 30 days by default; zero-retention agreements are available on enterprise plans).
- The scraper uses a bot-identifying User-Agent and respects standard HTTP semantics (no parallel hammering, 30-second timeout). It does not attempt to bypass paywalls or authentication.

Misuse

Risks and mitigations:

Risk	Mitigation
Generating fake docs to impersonate a library	Citations and confidence scoring make unverified outputs obvious.
Mass scraping of third-party sites	No built-in crawler – single-URL ingestion only.
Prompt injection via scraped content	System prompt runs first; user-visible citations expose injected content.
Leaking API keys in logs	Logger formats never include env vars; `.env` is gitignored.

Responsible Use Checklist

- [OK] Do ingest public documentation, your team's internal wikis, or source code you own.

- [OK] Do treat generated docs as drafts requiring human review.
- ☒ Don't ingest medical, legal, or financial documents and rely on the output without a qualified professional.
- ☒ Don't strip citations before sharing generated content – they're the user's audit trail.

Future Improvements

Future Improvements

Retrieval

- **Hybrid search.** Combine dense (embeddings) with sparse (BM25) for better recall on proper nouns and rare tokens.
- **Query rewriting.** A cheap rewrite step ("expand abbreviations, split compound questions") improves top-k hit rate without changing the retriever.
- **Re-ranking.** Add a cross-encoder re-ranker (e.g. `bge-reranker-base`) on the top-20 FAISS results before feeding top-k to the LLM.
- **Per-document deletion.** Currently `DELETE /knowledge-base` clears everything; add a `DELETE /knowledge-base/{doc_id}` route.

Prompt Engineering

- **Few-shot from synthetic data.** Export high-quality synthetic Q&A pairs and inject the top 2-3 as few-shot examples into the chat system prompt.
- **Self-consistency.** For high-stakes queries, call the LLM 3× with temperature 0.7 and pick the answer with the highest citation coverage.
- **Structured output.** Migrate doc generation to OpenAI's response-format JSON mode so the UI can render richer, sectioned documentation.

UX

- **Streaming responses.** Swap `/api/chat` for an SSE endpoint; the UI already has a code shape that supports incremental rendering.
- **Document explorer.** Click a citation in chat to jump to that chunk in the Knowledge Base viewer.
- **Saved sessions.** Persist conversation history in `localStorage` so refreshes don't lose context.

Evaluation

- Ship a proper **golden-set evaluation harness**: N synthetic Q&A pairs per ingested doc, measure retrieval hit@k and answer-contains-expected.
- Track **evaluation scores over time** in the metrics page.

Productionization

- **Auth.** Currently open; add a simple API-key guard or OAuth proxy.
- **Persistent vector store.** Swap FAISS on-disk for Qdrant / pgvector for multi-process deployments.
- **Docker Compose.** Single `docker compose up` to run backend + frontend.
- **Observability.** Replace in-memory metrics with Prometheus + Grafana.